

OppNet Chat - Relatório de Projeto



Realizado Por:
Diogo Tavares (I58049),
Francisco Rodrigues (I59119)

Unidade Curricular: Sistemas Móveis e Aplicações

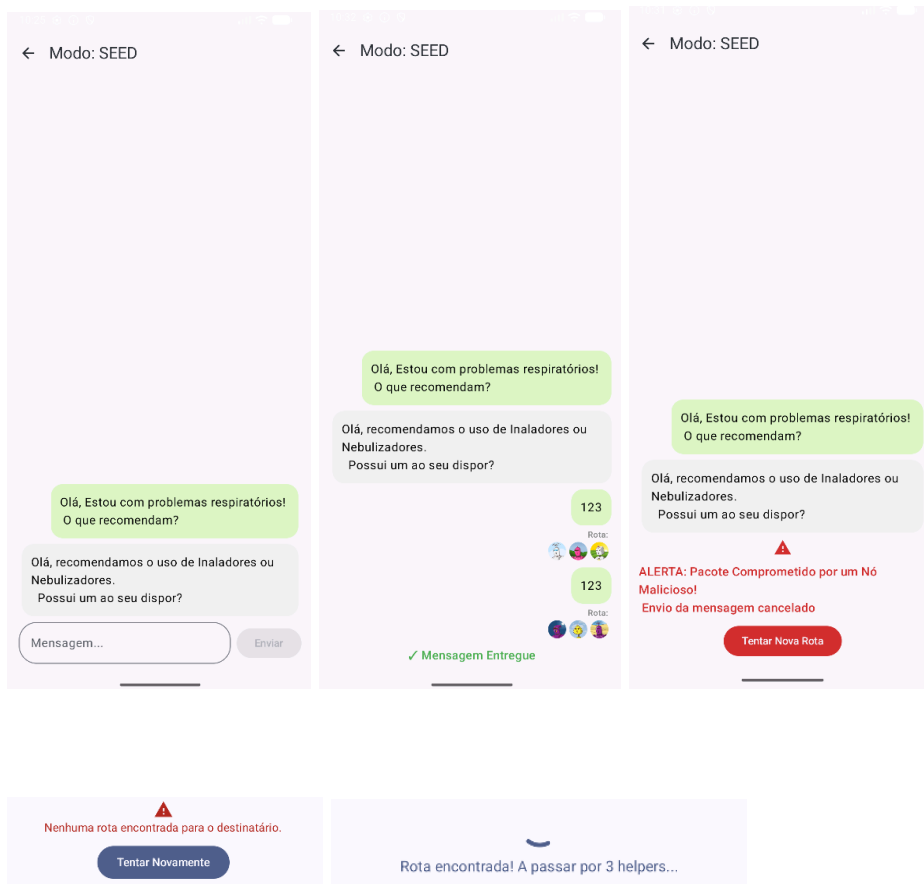
1. Descrição da Aplicação

O OppNet Chat é uma aplicação sobre o tema de Redes Oportunistas em que temos dois tipos de User Role (Cargo de Utilizador), o tipo Helper em que será o que transporta os pacotes e o tipo Seed em que é aquele que envia a mensagem (pacote) para depois ser enviada através dos helpers até chegar ao destinatário.

Destinadas a pessoas comuns com falta de conectividade em situações críticas ou de emergência como por exemplo tempestades, as Redes

Oportunistas (OppNets) são usadas para mensagens de emergência resilientes.

A aplicação também acolhe exemplos das principais ameaças, os Ataques Passivos e Ataques Ativos, como por exemplo EavesDropping, BlackHole Attack, e a modificação de pacotes.



2. Funcionalidades Principais:

- Simulação do Fluxo de Envio de Mensagens
- Encriptação de Mensagens
- Simulação de Tentativas Ataque Passivo
- Simulação de Tentativas de Ataque Ativo
- Implementação de API para criar imagens anónimas para os helpers
- Simulação de Reencaminhamento de Mensagens (Store & Forward) como helper

3. Mapping - Conceitos Lecionados e Desenvolvimento

Unidade do Curso	Conceitos Lecionados	Implementação no Projeto (OppNet Guard)
Unidade 1: Seu primeiro app	Fundamentos de UI (Layouts Básicos, Modificadores) e Sintaxe Kotlin.	Estruturação dos ecrãs usando <code>Column</code> , <code>Row</code> , <code>Box</code> e <code>Spacer</code> para alinhamento. Utilização intensiva do <code>Modifier</code> (ex: <code>.padding()</code> , <code>.fillMaxSize()</code>) para organizar o ecrã inicial e de chat.
Unidade 2: Criar a interface	Interatividade, Inputs de Utilizador e Elevação de Estado (<i>State Hoisting</i>).	Utilização de <code>OutlinedTextField</code> para a escrita de mensagens e <code>Button</code> para ações. Passagem de funções lambda (ex: <code>onSendMessage</code> , <code>onRoleSelected</code>) dos ecrãs filhos para os pais, respeitando a Elevação de Estado.

Unidade 3: Mostrar listas	Material Design 3, Temas (<code>Theme</code>) e Listas Dinâmicas (<code>LazyColumn</code>).	Uso de <code>LazyColumn</code> para desenhar o histórico de mensagens e a lista de pacotes no modo <i>Helper</i> . Implementação de <code>Scaffold</code> , <code>TopAppBar</code> , <code>Card</code> e uso dos ficheiros <code>Color.kt</code> e <code>Theme.kt</code> para garantir uma interface padronizada.
Unidade 4: Arquitetura	Arquitetura de Software (<code>ViewModel</code> , <code>StateFlow</code>) e Navegação (<code>NavHost</code>).	Separação da lógica de UI usando <code>HomeViewModel</code> e <code>ChatViewModel</code> . Definição do estado com <code>data classes</code> (<code>ChatUIState</code>). Implementação do <code>NavGraph</code> para navegar entre ecrãs, passando o argumento <code>{role}</code> (SEED/HELPER).
Unidade 5: Ligar à Internet	Corrotinas (Trabalho assíncrono), <i>Coil</i> (Imagens) e Estados de Rede.	Uso de <code>viewModelScope.launch</code> e <code>delay()</code> para simular atrasos de rede. Criação da <i>sealed interface</i> <code>NetworkState</code> para gerir carregamentos/erros. Uso do <code>AsyncImage</code> (<i>Coil</i>) para renderizar os avatares (Robohash) dos robôs na interface.

4. Obstáculos Encontrados

- Tivemos dificuldades em achar um bom uso de API's e ficamos pela geração de imagens para helpers.
- Achar uma maneira simples e eficiente de mostrar mensagens encriptadas de exemplo para o helper.
- Demonstrar exemplos do fluxo da aplicação em casos de ataques ativos e passivos(usamos o exemplo encriptado e a probabilidade aleatória de uma mensagem ser modificada)
- Demonstrar o número de helpers sem os identificar(uso de imagens da API)
- Exemplificar o feedback de pacotes reencaminhados como helper sem aceder aos pacotes(lista de pacotes gerados aleatoriamente e exemplo de mensagem encriptada no acesso)

5. Objetivos Atingidos

O desenvolvimento do protótipo OppNet Guard permitiu aplicar de forma prática os conhecimentos do curso Android Basics with Compose, alcançando com sucesso os seguintes objetivos:

Arquitetura e Interface (Unidades 1 a 4):

Implementação de uma arquitetura robusta (separando a lógica da UI com ViewModels e StateFlow) e construção de uma interface responsiva em Jetpack Compose, utilizando o Material Design 3 e listas dinâmicas eficientes (LazyColumn).

Navegação Dinâmica (Unidade 4): Criação de um sistema de navegação fluido (Compose Navigation) que transita e adapta a aplicação consoante o papel selecionado pelo utilizador no ecrã inicial (SEED ou HELPER).

Simulação de Rede e Integração de APIs (Unidade 5): Utilização de Corrotinas em segundo plano para emular com realismo a latência de envio de mensagens, e integração da biblioteca Coil para consumir a API externa do Robohash e renderizar avatares de forma assíncrona.

Demonstração de Redes Oportunistas e Segurança: Simulação prática do paradigma Store-and-Forward (com receção de pacotes em background). Isto permitiu demonstrar visualmente ameaças reais, como ataques passivos e ataques ativos.